

DALIL

a buslahub product · dalil.buslahub.com

Functional Specification

Full-Stack SaaS Platform for Creative Agencies

Version	2.0 (derived from PRD v1.6)
Status	Finalized — Ready for Implementation
Date	June 2026
PRD Reference	Dalil PRD v1.6
Audience	Engineering, QA, Design
Classification	CONFIDENTIAL

This document translates the Dalil PRD into precise behavioural contracts, state machines, validation rules, and data-flow specifications for engineers and QA.

Table of Contents

1. Introduction & Scope

This Functional Specification (FS) translates the Dalil PRD v1.6 into precise engineering contracts. Where the PRD describes what the system must do, this document describes exactly how each behaviour is triggered, validated, persisted, and communicated. Engineers and QA should treat this as the primary reference for implementation decisions.

1.1 Purpose

- Define the behaviour of every user-facing action as a testable contract.
- Specify all state machines, transitions, and guards.
- Enumerate validation rules and error responses for every input.
- Document the data-flow between modules (who writes, who reads, what triggers what).
- Provide the authorization matrix cross-referencing every action with its required permission key.

1.2 Document Conventions

Convention	Meaning
MUST	A hard requirement. Any deviation is a defect.
MUST NOT	Prohibited. Any occurrence is a defect.
SHOULD	Recommended. Deviations require documented justification.
MAY	Optional. Implement if time allows.
[PK]	Permission key required by the caller.
[SYS]	System-triggered action (BullMQ, cron, WebSocket).
FSM	Finite State Machine — a status field with defined transitions.

1.3 Relationship to PRD

Every section of this FS maps to one or more sections of the PRD v1.6. Where a PRD decision is a v1 scoped item, that scope is preserved here. Deferred features are referenced but not specified.

1.4 Out of Scope (Deferred)

- Time tracking (logged hours, timers) — entire feature deferred.
- Capacity planning (per-member availability, utilization) — deferred.
- Advanced analytics (velocity, bottleneck, revenue trends) — UI deferred; TransitionLog data capture is in scope.
- Gateway payment reconciliation (Moyasar/Tap/HyperPay) — manual recording only in v1.
- E-signature — authenticated Accept action + audit log serves as acceptance record.
- Native mobile app — responsive web only at launch.
- Self-serve tenant onboarding — assisted onboarding for early tenants; self-serve is Phase 8.

1.5 Terminology Consistency Reference

(Added for Engineering Completeness)

The following table is the authoritative glossary for all entity names, status enums, and permission keys used throughout this document. Any deviation in implementation is a defect.

Term	Canonical Form	Never Use
Tenant user role model	Permission Group	Role, UserRole, RoleGroup
Authorization check	permission key (e.g. quotations.create)	role name, role check
Client returning login	AuthToken { type: LOGIN }	login link, re-entry token
Client first-time link	AuthToken { type: ONBOARDING }	magic link (informal only), onboarding link
Invoice paid status	PAID	Paid, paid, COMPLETE
Phase blocked status	PENDING	LOCKED, BLOCKED, WAITING
Phase work in progress	ACTIVE	IN_PROGRESS (reserved for Task)
Task work in progress	IN_PROGRESS	ACTIVE (reserved for Phase)
Project awaiting start	PENDING_START	NOT_STARTED, DRAFT
Blueprint service set	blueprintServiceSet	serviceSet, serviceList, phaseServices
Proposal revision cycle	amendment loop	review loop, change loop
Billing checkpoint	QuotationMilestone	Milestone alone (always qualified)
Phase billing grouping	MilestonePhase (link table)	phase-milestone, milestone-phase
Client auth scope	Client JWT	portal token, client token

2. Actor Model & Authentication

Dalil has three distinct actor types. Each has its own authentication path, session model, and authorization mechanism. These **MUST** never be conflated in implementation.

2.1 Actor Summary

Actor	Scope	Auth Mechanism	Session	Authorized By
Platform Owner	Platform-level	Email + password (bcrypt/argon2)	JWT RS256, 15-min access + 7-day refresh rotation	Hardcoded platform role
Tenant User	Tenant-level	Email + password (bcrypt/argon2), forgot-password token	JWT RS256, 15-min access + 7-day refresh rotation	Permission Groups (RBAC)
Client	Portal-level	Magic link (onboarding, 7d) OR login link (15–30 min) OR email+password (optional)	Short-lived JWT scoped to contactId + tenantId	Client Portal auth — no tenant RBAC

2.2 Tenant User Authentication

2.2.1 Login

1. Client posts { email, password } to POST /api/v1/auth/login.
2. System looks up User by tenantId (resolved from subdomain/header) + email.
3. If user not found OR isActive=false → 401 { code: 'INVALID_CREDENTIALS' } (same message for both — no user enumeration).
4. Compare password with bcrypt/argon2 hash. On mismatch → 401 same code.
5. On success: generate access token (RS256, 15 min, payload: { sub: userId, tenantId, type: 'tenant_user' }) and refresh token (opaque, stored hashed, 7-day TTL).
6. Return { accessToken, refreshToken, expiresIn: 900 }.

2.2.2 Token Refresh

7. Client posts { refreshToken } to POST /api/v1/auth/refresh.
8. System verifies refresh token hash exists, not expired, not revoked.
9. If invalid → 401 { code: 'INVALID_REFRESH_TOKEN' }.
10. On success: rotate — invalidate old refresh token, issue new access + refresh token pair.

2.2.3 Logout

11. POST /api/v1/auth/logout with Bearer token.
12. Revoke refresh token from store. Access token remains valid until natural expiry (stateless).

2.2.4 Forgot Password

13. POST /api/v1/auth/forgot-password { email }.

14. System always responds 200 (no user enumeration). If user exists: generate single-use reset token (hashed, 1-hour TTL), email link.
15. POST /api/v1/auth/reset-password { token, newPassword }.
16. Validate token. Validate password (min 8 chars, 1 upper, 1 digit). Hash and save. Invalidate all refresh tokens for this user.

2.3 Client Authentication

2.3.1 Onboarding Magic Link

17. System generates AuthToken { type: ONBOARDING, contactId, tokenHash, expiresAt: +7d, usedAt: null }.
18. One-time link: GET /api/v1/auth/onboarding/verify/:rawToken.
19. System hashes rawToken, looks up AuthToken. Rejects if: not found, type ≠ ONBOARDING, usedAt is set, expiresAt is past.
20. On success: mark usedAt=now. Create Client session JWT (payload: { sub: contactId, tenantId, type: 'client' }). Redirect to portal account-creation form.
21. Client submits account form (name, optional password). System creates Contact.passwordHash if password provided.

2.3.2 Passwordless Login Link

22. Client POSTs { email } to POST /api/v1/auth/client-login/send.
23. System always responds 200. If Contact exists for this tenant: generate AuthToken { type: LOGIN, expiresAt: +20min, usedAt: null }.
24. Email delivers link: GET /api/v1/auth/client-login/verify/:rawToken.
25. System validates: exists, type=LOGIN, not used, not expired.
26. On success: mark usedAt=now. Issue Client JWT (30-day session or configurable). Return { accessToken }.

2.3.3 Optional Password Login

27. After first sign-in, client MAY set password via POST /api/v1/auth/client/set-password { newPassword }.
28. System saves bcrypt/argon2 hash on Contact. Passwordless login remains available.
29. Client can subsequently POST /api/v1/auth/client/login { email, password }.

2.4 Authorization Resolution (RBAC)

2.4.1 Effective Permissions Algorithm

On every protected tenant-user request, the permission guard executes:

30. Resolve tenantId from JWT claim. Verify User.tenantId matches.
31. Fetch all UserPermissionGroup rows for userId.
32. Fetch all PermissionGroupPermission rows for those groupIds.
33. Collect the union of all permission.key values → base set.
34. Fetch UserPermissionOverride rows for userId.
35. For each override: GRANT → add key to set; REVOKE → remove key from set.
36. effectivePermissions = resulting set.

37. Guard checks: if `requiredPermission ∈ effectivePermissions` → allow; else → 403 { code: 'FORBIDDEN', requiredPermission }.

Performance Note

Cache effective permissions per `userId` in Redis with TTL 60s.
 Invalidate cache on: `UserPermissionGroup` change, `PermissionGroupPermission` change, `UserPermissionOverride` change.
 Cache key: `perm:{tenantId}:{userId}`

2.4.2 Administrator Protection Invariants

These checks MUST run before any mutation to `UserPermissionGroup`, `User.isActive`, or `PermissionGroupPermission` where the Administrator group is involved:

- MUST NOT: Delete `PermissionGroup` where `isSystem=true`.
- MUST NOT: Remove a user from Administrator group if they are the last member.
- MUST NOT: Deactivate a user who is the sole Administrator-group member.
- MUST NOT: Revoke `users.manage`, `permission-groups.manage`, or `settings.manage` from the Administrator `PermissionGroup`.

Any of the above attempts MUST return 422 { code: 'ADMIN_PROTECTION_VIOLATION', reason: '<specific reason>' }.

2.5 Token Security & Replay Prevention

(Added for Engineering Completeness)

- All `AuthToken` values are stored as SHA-256 hashes. The raw token is never persisted.
- On verification: hash the incoming `rawToken`, look up by hash. Time-safe comparison MUST be used.
- `usedAt` is set atomically in the same transaction as the action it authorises (account creation or session issuance). A token found with `usedAt` set MUST always return 401 `AUTH_TOKEN_INVALID` — no re-use under any condition.
- Parallel onboarding link requests: if a second `ONBOARDING` token is generated while a valid one already exists, the old token MUST be invalidated (deleted, or `usedAt` set to now) before the new one is persisted. Only one valid `ONBOARDING` token per `contactId` per tenant at any time.
- `ONBOARDING` tokens are not interchangeable with `LOGIN` tokens. Guard MUST verify `type === expected type` before accepting.

2.6 Concurrent Session Handling

(Added for Engineering Completeness)

- Tenant Users: multiple concurrent sessions are permitted. Each refresh token is independent. Logout revokes only the presented refresh token. A 'logout all devices' action (`PATCH /api/v1/auth/logout-all`) MUST revoke all refresh tokens for the `userId`.
- Clients: same model — concurrent portal sessions permitted. Login links do not invalidate prior sessions.
- Platform Owner: same JWT model as Tenant Users.
- Deactivated users (`isActive=false`): existing valid access tokens remain valid until expiry (max 15 min). Refresh token lookup MUST check `User.isActive` and return 401 if false, preventing new access token issuance.

3. Multi-Tenancy & Tenant Lifecycle

3.1 Tenant Isolation

- Every database query for tenant-scoped entities MUST include WHERE tenantId = :tenantId.
- Prisma middleware MUST inject tenantId on every query. Any query without tenantId on a scoped model is a defect.
- S3 paths MUST follow pattern: tenant/{tenantId}/... No cross-tenant path access is ever permitted.
- WebSocket rooms are namespaced per tenant: tenant:{tenantId}.

3.2 Tenant Bootstrap Sequence

When a new tenant is provisioned (by Platform Owner via POST /api/v1/tenants):

38. Create Tenant { name, slug, planId, subscriptionStatus: ACTIVE }.
39. Create first Tenant User { tenantId, email, name, isActive: true }.
40. Create default PermissionGroups for tenant (see Section 9.2).
41. Assign first user to Administrator PermissionGroup via UserPermissionGroup.
42. Send welcome email with login credentials.

All five steps execute in a single database transaction. Partial bootstrap is a defect.

3.3 Tenant Identification

- Tenant is resolved from: (a) subdomain of the request host (e.g. agency1.dalil.buslahub.com), or (b) X-Tenant-Slug header for API clients.
- Tenant slug lookup MUST be case-insensitive.
- If tenant not found → 404 { code: 'TENANT_NOT_FOUND' }.
- If tenant subscriptionStatus = SUSPENDED → 403 { code: 'TENANT_SUSPENDED' }.

3.4 Plan Entitlement Enforcement

At runtime, feature access is checked against the tenant's plan entitlements:

- Fetch Tenant.planId → Plan → Entitlement rows.
- Cache entitlement map in Redis: entitlement:{tenantId}, TTL 300s. Invalidate on plan change.
- Active project count: COUNT(Project WHERE tenantId AND status NOT IN [COMPLETED, ARCHIVED, CANCELLED]).
- If creating a project would exceed max_projects entitlement → 422 { code: 'PLAN_LIMIT_EXCEEDED', limit: 'max_projects', current, max }.
- User invite: check max_users entitlement before creating User.
- A project in 'not yet started' state (accepted proposal, Start not yet clicked) does NOT count against the active project limit.

3.5 Soft-Delete & Archival Strategy

(Added for Engineering Completeness)

Dalil does not hard-delete most entities. The table below defines the deletion/archival strategy per entity type.

Entity	Strategy	Effect on References
Service	Archive (isArchived=true)	Existing Blueprint/Package references preserved. Archived services excluded from new selections.
SalesFlowStage	Hard delete	Leads in deleted stage cascade to previous stage in same transaction.
Blueprint	Soft delete (isDeleted=true)	Running projects unaffected (snapshot model). Blueprint excluded from quotation filter.
PermissionGroup	Hard delete (non-system)	UserPermissionGroup rows cascade deleted. Cache invalidated for all affected users.
User	Deactivate (isActive=false)	Tasks remain assigned. Notifications continue. Cannot log in.
Task	Hard delete	Only when entire ProjectPhase is deleted (rare, admin only).
Proposal	Status-only (REJECTED/REOPENED)	Never physically deleted. Audit trail preserved.
Invoice	No deletion permitted	Corrections via offsetting Payment or ChangeRequest.
TransitionLog	Append-only, no deletion	Immutable by design.
Approval	Append-only, no deletion	Immutable by design.

3.6 Idempotency Considerations

(Added for Engineering Completeness)

- POST /api/v1/projects/:id/start: idempotent — if project.status is already ACTIVE, return 200 with current project state (no re-snapshot, no re-invoice). Guard logs a warning.
- POST /api/v1/proposals/:id/accept: idempotent — if already ACCEPTED, return 200. Do not re-trigger invoice or notification.
- POST /api/v1/auth/onboarding/send: rate-limited to 3 requests per contact per hour. Subsequent requests within the hour return 429 RATE_LIMITED.
- BullMQ jobs: generate-invoice jobs MUST be deduplicated by milestoneId. If a job for a milestoneId is already in queue or completed, do not enqueue a second. Use job ID = 'invoice: {milestoneId}'.

4. Services & Packages Module

4.1 Service Management

4.1.1 Create Service [PK: services.manage]

POST /api/v1/services

Field	Type	Validation
name	string	Required. 2–100 chars. Unique per tenant (case-insensitive).
description	string?	Optional. Max 1000 chars.
defaultPrice	decimal	Required. ≥ 0 . Max 2 decimal places. Stored in SAR.

On success: 201 { service }. On duplicate name: 422 { code: 'DUPLICATE_SERVICE_NAME' }.

4.1.2 Update Service [PK: services.manage]

PATCH /api/v1/services/:id — partial update. Any field may be updated. Name uniqueness re-validated. Updating defaultPrice does NOT retroactively affect existing quotations (quotations snapshot the price at creation time).

4.1.3 Delete Service [PK: services.manage]

DELETE /api/v1/services/:id

- MUST check: if service is referenced by any Phase (blueprint or projectPhase), or any PackageService row → return 422 { code: 'SERVICE_IN_USE', referencedBy: ['blueprints', 'packages'] }.
- If in use → archive instead: set service.isArchived=true. Archived services do not appear in new-service selection but existing references remain valid.
- If not referenced → hard delete permitted.

4.2 Package Management

4.2.1 Create / Update Package [PK: packages.manage]

POST /api/v1/packages | PATCH /api/v1/packages/:id

Field	Type	Validation
name	string	Required. 2–100 chars. Unique per tenant.
bundledPrice	decimal	Required. ≥ 0 . Authoritative price — NOT derived from service prices.
serviceIds	uuid[]	Required. Min 2 services. All must belong to same tenant and not be archived.

Package price is authoritative. At quotation time, selecting a package sets quotation.totalAmount = package.bundledPrice (unless admin overrides). The price MUST NOT be re-derived from constituent service defaultPrices.

4.2.2 Package Expansion at Quotation

When a package is selected in the quotation builder, the system resolves: `selectedServices = package.serviceIds` (via `PackageService` join). This set is used for exact-match blueprint filtering. The package's `serviceId` list is immutable for any given quotation once confirmed.

4.3 Edge Cases — Services & Packages

(Added for Engineering Completeness)

4.3.1 Concurrent Service Edits

- If two users submit `PATCH /services/:id` simultaneously, last-write-wins is acceptable for non-critical fields (name, description).
- `defaultPrice` updates: no optimistic locking required in v1. Engineers **SHOULD** note this is a known eventual-consistency window.

4.3.2 Package with Archived Service

- A package **MUST NOT** be created with an archived service. Validation: check `isArchived=false` for every `serviceId`.
- If a service is archived after being added to a package: the package becomes stale. It **MUST** be flagged (`package.hasArchivedService = computed on read`). Selection of a stale package in the quotation builder **MUST** be blocked with 422 { code: `'PACKAGE_CONTAINS_ARCHIVED_SERVICE'`, `serviceIds: [...]` }.

4.3.3 Single Service in Package

- Min 2 services per package is enforced (PRD: a package is a bundle). Attempt to create single-service package → 422 { code: `'PACKAGE_TOO_FEW_SERVICES'`, `min: 2` }.

5. CRM & Lead Management Module

5.1 Sales Flow Configuration [PK: crm.edit]

PATCH /api/v1/sales-flow/stages — replaces the entire stage list in a single transaction.

- Stages are ordered; order field is 1-based integer.
- Min 1 stage, max 20 stages per tenant.
- Each stage: name (2–50 chars), order (unique integer).
- COMPLETED and ARCHIVED are system pseudo-stages, not stored as SalesFlowStage rows. They are always present at the end of the pipeline view.
- If a stage is removed and leads exist in that stage: those leads move to the previous stage. If the first stage is removed, they move to the new first stage. This cascades in the same transaction.

5.2 Lead Lifecycle FSM

5.2.1 Lead Status

Lead.conversionStatus is a FSM:

State	Description	Allowed Transitions
OPEN	Active in pipeline	→ CONVERTED (crm.edit), → STOPPED (crm.edit)
CONVERTED	Won; client created	→ OPEN (if re-opened; rare)
STOPPED	Lost or archived	→ OPEN (if re-opened)

5.2.2 Stage Movement [PK: crm.edit]

43. PATCH /api/v1/leads/:id/stage { stageId }
44. Validate: stageId belongs to this tenant.
45. Update Lead.stageId.
46. Write TransitionLog { entityType: LEAD, entityId: leadId, fromState: oldStageId, toState: newStageId, at: now, byUserId }.

5.2.3 Conversion Flow [PK: crm.edit]

47. PATCH /api/v1/leads/:id/convert { decision: 'YES' | 'NO' }
48. If NO: set conversionStatus=STOPPED. No further action. TransitionLog written.
49. If YES:
 - Set Lead.conversionStatus=CONVERTED.
 - Check if Contact already has a client account (Contact.accountCreatedAt is set).
 - If no account: generate AuthToken { type: ONBOARDING, expiresAt: +7d }. Email magic link. Return 200 { status: 'ONBOARDING_LINK_SENT' }.
 - If account exists: return 200 { status: 'EXISTING_CLIENT', contactId }. Admin can immediately create a quotation.

5.3 Contact Profile

A Contact is a single global identity keyed by email. A Contact can belong to multiple tenant portals. GET /api/v1/contacts/:id/history returns:

- All leads for this contact on this tenant.
- All proposals linked to this contact.
- All projects linked to this contact.
- Activity log entries (from TransitionLog and approval records).

[PK: contacts.view] for history; [PK: contacts.manage] for create/update.

5.4 CRM Edge Cases

(Added for Engineering Completeness)

5.4.1 Duplicate Contact Email

- Contact.email is globally unique (across all tenants for a given email). Attempting to create a second Contact with the same email within the same tenant → 422 { code: 'DUPLICATE_CONTACT_EMAIL' }.
- The same email CAN exist across different tenants (the same person can be a client of multiple agencies).

5.4.2 Lead Assignment to Inactive User

- Assigning a lead to a deactivated user (isActive=false) → 422 { code: 'USER_INACTIVE', userId }.
- If a user is deactivated while leads are assigned: leads remain assigned. A notification is sent to users with crm.edit to reassign.

5.4.3 Re-opening a STOPPED Lead

- PATCH /api/v1/leads/:id { conversionStatus: 'OPEN' } is permitted (transition STOPPED → OPEN). Write TransitionLog entry. This does NOT re-activate any prior onboarding link.

5.4.4 Converting an Already-Converted Lead

- If Lead.conversionStatus is already CONVERTED and PATCH /leads/:id/convert is called again → return 422 { code: 'LEAD_ALREADY_CONVERTED' }. Do not generate a second onboarding link.

5.4.5 Deleting a Stage with Active Leads

- Stage deletion with active leads requires explicit confirmation from the caller (force: true in body). Without force:true → 422 { code: 'STAGE_HAS_ACTIVE_LEADS', count }. With force:true → proceed with cascade.

6. Quotation & Proposal Module

6.1 Quotation Builder

6.1.1 Create Quotation [PK: quotations.create]

POST /api/v1/quotations

Field	Type	Validation Rule
contactId	uuid	Required. Contact must belong to this tenant.
serviceIds	uuid[]?	Either serviceIds OR packageId is required, not both.
packageId	uuid?	If provided: expand to package.serviceIds. Either this or serviceIds.
blueprintId	uuid	Required. Must exactly match the selected/expanded service set (see 6.2).
paymentModel	enum	Required. ONE_TIME_BEFORE_START ONE_TIME_AFTER_FINISH MILESTONE_BASED.
totalAmount	decimal?	Optional override. If omitted, system computes default (see 6.3).
milestones	array?	Required if paymentModel=MILESTONE_BASED. See 6.4.

6.2 Exact-Match Blueprint Filter

Before a blueprint can be selected, the system filters available blueprints:

50. Resolve selectedServiceSet: if serviceIds provided → that set; if packageId → package's serviceIds.
51. For each blueprint owned by this tenant: compute blueprintServiceSet = { phase.serviceId for all phases of blueprint }.
52. Filter: include blueprint if and only if blueprintServiceSet === selectedServiceSet (set equality — same elements, ignoring order, no extras, no missing).
53. Return matching blueprints. If result is empty → return { matches: [], fallback: 'NO_BLUEPRINT_FOUND', blueprintEditorUrl: '/blueprints/new' }.

Exact-Match Rule

Selecting services {A, B} returns ONLY blueprints with service set = {A, B}.
 A blueprint with phases covering {A, B, C} is NOT returned.
 A blueprint with phases covering only {A} is NOT returned.
 Duplicate service in multiple phases: {A, A, B} → service set is {A, B} (set deduplication).

6.3 Total Amount Computation

System computes default totalAmount as follows:

- If individual services selected: totalAmount = SUM(service.defaultPrice for each selected service).
- If package selected: totalAmount = package.bundledPrice (NOT re-derived from service prices).
- Admin may override totalAmount by submitting a custom value. Record totalOverridden=true.
- Any milestone amounts must sum to the final totalAmount (whether default or overridden).

6.4 Milestone Grouping & Validation

6.4.1 Milestone Structure

Each QuotationMilestone groups one or more blueprint phases. A one-time payment is modeled as a single milestone covering ALL phases.

6.4.2 Dependency-Closed Validation

Before saving milestones, the system **MUST** validate the dependency-closed invariant for each milestone:

54. For each milestone M, collect its phaselds → S.
55. For each pair of phases (p1, p2) in S: find all phases on any dependency path from p1 to p2 in the blueprint's DAG.
56. Every such intermediate phase **MUST** also be in S.
57. If violation found → 422 { code: 'MILESTONE_NOT_DEPENDENCY_CLOSED', milestoneId, offendingPhaselds }.

6.4.3 Additional Milestone Validations

- Every phase in the blueprint **MUST** appear in exactly one milestone. Missing phase → 422 { code: 'PHASE_NOT_IN_MILESTONE', phaseld }. Duplicate → 422 { code: 'PHASE_IN_MULTIPLE_MILESTONES', phaseld }.
- SUM(milestone.amount) **MUST** = quotation.totalAmount. Tolerance: ±0.01 SAR for rounding. Violation → 422 { code: 'MILESTONE_AMOUNTS_DONT_SUM', expected, actual }.
- Each milestone.amount ≥ 0.

6.5 Quotation Status FSM

Status	Description	Transitions
DRAFT	Being built	→ SENT (send proposal)
SENT	Awaiting client response	→ AMENDMENT_REQUESTED, → ACCEPTED, → REJECTED
AMENDMENT_REQUESTED	Client requested changes	→ SENT (admin revises and resends)
ACCEPTED	Client accepted	→ PROJECT_STARTED (when Start clicked)
REJECTED	Client rejected	→ REOPENED (admin revises)
REOPENED	Revised after rejection	→ SENT (on resend)

6.6 Proposal Generation & Lifecycle

6.6.1 Generate Proposal [PK: quotations.create]

POST /api/v1/quotations/:id/generate-proposal — generates a PDF proposal from quotation data. Creates Proposal { quotationId, versionNo: (max existing + 1 or 1), status: DRAFT }. PDF stored in S3 at: tenant/{tenantId}/proposals/{proposalId}/proposal_v{versionNo}.pdf.

6.6.2 Send Proposal [PK: quotations.send]

POST /api/v1/quotations/:id/send-proposal

58. Validate quotation status is DRAFT or AMENDMENT_REQUESTED or REOPENED.
59. Set Proposal.status = SENT, Proposal.sentAt = now.
60. Set Quotation.status = SENT.
61. Email proposal PDF to Contact.email with portal link.
62. Portal notification created for client.

6.6.3 Client Amendment Request [Client actor]

POST /api/v1/proposals/:id/amend { comment }

63. Set Proposal.status = AMENDMENT_REQUESTED. Append amendment to Proposal.amendments[].
64. Set Quotation.status = AMENDMENT_REQUESTED.
65. In-app notification + email to users with quotations.create permission.

6.6.4 Client Accept [Client actor]

POST /api/v1/proposals/:id/accept

66. Set Proposal.status = ACCEPTED. Set Quotation.status = ACCEPTED.
67. If paymentModel = ONE_TIME_BEFORE_START: enqueue invoice generation job (fires when project starts).
68. Notify users with projects.start permission via in-app + email.

6.6.5 Client Reject [Client actor]

POST /api/v1/proposals/:id/reject { reason? }

69. Set Proposal.status = REJECTED. Set Quotation.status = REJECTED.
70. Notify users with quotations.create permission.
71. Admin may revise quotation and create new proposal version → status moves to REOPENED → SENT on resend.

6.7 Quotation & Proposal Edge Cases

(Added for Engineering Completeness)

6.7.1 Editing a Quotation After Sending

- A SENT quotation MUST NOT be directly edited. The amendment loop (Section 6.6.3) is the only path: client requests amendment → admin creates new quotation version.
- If admin attempts PATCH /quotations/:id while status=SENT → 422 { code: 'QUOTATION_NOT_EDITABLE', status }.
- An ACCEPTED quotation is permanently immutable. Status transitions beyond ACCEPTED are: → PROJECT_STARTED (system).

6.7.2 Blueprint Changed After Quotation Created

- Quotation records blueprintId as a reference. If the blueprint is edited or deleted after quotation creation but before project Start, the quotation remains valid — it references the blueprint's phases at snapshot time.

- If the blueprint is deleted: quotation.blueprintId is retained for reference. The Start action will fail with 422 { code: 'BLUEPRINT_DELETED' } — admin must create a new quotation.

6.7.3 Concurrent Proposal Acceptance

- If two portal sessions attempt POST /proposals/:id/accept simultaneously: the first succeeds. The second finds status=ACCEPTED and returns idempotent 200 (see Section 3.6).

6.7.4 Milestone Amount Rounding

- All amounts are DECIMAL(12,2). When splitting a non-even total across milestones, the admin is responsible for ensuring sum = total. The system applies a ± 0.01 tolerance. Engineer note: store amounts as integer cents internally if precision is a concern.

6.7.5 Zero-Amount Milestones

- A milestone with amount=0 is permitted (e.g. a discovery phase included at no charge). It still triggers an invoice generation job (invoice.amount=0). The email template should handle zero-amount invoices gracefully (show 'No charge due').

6.7.6 Proposal Version Numbering

- versionNo is 1-based, sequential per quotation, never reused. If a version is created and then rejected, the next version is versionNo + 1, not a replacement of the rejected one.
- At most one Proposal per quotation may be in an actionable status $\in \{\text{DRAFT, SENT, AMENDMENT_REQUESTED, REOPENED}\}$ at any time. The proposal with the highest versionNo is the active proposal; earlier versions are historical and retain their terminal status (REJECTED) or are non-actionable. The system MUST NOT present two live (actionable) proposals simultaneously. No new status is introduced — this invariant is enforced using the existing proposal status enum from the PRD.

7. Blueprint & Board Configuration Module

7.1 Blueprint Creation & Editing [PK: blueprints.manage]

POST /api/v1/blueprints | PATCH /api/v1/blueprints/:id

Field	Type	Validation
name	string	Required. 2–100 chars. Unique per tenant.
description	string?	Optional. Max 500 chars.
phases	Phase[]	Required. Min 1 phase.

7.1.1 Phase Validation Rules

- Each phase: name (2–100 chars), serviceId (must exist and not be archived in same tenant), order (1-based integer, unique within blueprint), boardId.
- Phase dependencies: dependsOnPhaseIds[] — each id must be another phase in the same blueprint. No circular dependencies (DAG enforced).
- Circular dependency detection: topological sort on phase DAG. If sort fails → 422 { code: 'CIRCULAR_PHASE_DEPENDENCY', cycle }.
- boardId: must reference a TaskBoard belonging to this tenant.

7.2 Blueprint Service Set Derivation

blueprintServiceSet = new Set(phases.map(p => p.serviceId))

This is computed on read (not stored). Used exclusively for exact-match filtering at quotation time.

7.3 Task Board Management [PK: blueprints.manage]

7.3.1 Create Board

POST /api/v1/task-boards { name, columns: [{ name, order }] }

- Min 1 column. Max 10 columns per board.
- Column names: 2–50 chars. Unique within board.

7.3.2 Assign Board to Phase

POST /api/v1/task-boards/:boardId/assign-phase { phaseId } — links an existing board to a phase. A phase can be reassigned to a different board by a new assignment (previous is overwritten). Multiple phases can share one board.

7.3.3 Task Templates

Task templates are pre-defined cards in board columns. On project Start, each template becomes a real Task in the corresponding ProjectPhase. Templates: { title, description?, columnId }.

7.4 Blueprint Snapshot-on-Start

When a user with projects.start clicks Start:

72. For each Phase in blueprint: create ProjectPhase row copying { name, serviceId, boardId, dependsOnPhaseIds, order } plus new fields { projectId, status: PENDING, progress: 0, deadline: (phase.defaultDurationDays ? startDate + defaultDurationDays : null) }.
73. For each task template on the board: create Task row in the ProjectPhase { title, description, columnId, status: TODO, projectPhaseId }.
74. After snapshot: all reads for project structure use ProjectPhase rows, never the original Blueprint.Phase rows.
75. Blueprint.phases may be edited after Start without affecting running projects.

7.5 Blueprint Immutability Guarantee

Snapshot Contract

Once a project is started, its structure (phases, board, dependencies) is governed exclusively by ProjectPhase rows.
 Blueprint edits MUST NOT touch any ProjectPhase row.
 blueprint.id on a Project is an origin reference for analytics only — it is never used to resolve live project structure.

7.6 Blueprint Edge Cases

(Added for Engineering Completeness)

7.6.1 Editing a Blueprint Used by Active Projects

- A blueprint CAN be edited at any time regardless of linked projects. The snapshot-on-Start guarantee (Section 7.5) ensures running projects are unaffected.
- The blueprint editor MUST display a warning when the blueprint is linked to active projects: 'X active projects use this blueprint. Edits will not affect running projects.'

7.6.2 Deleting a Blueprint with Linked Projects

- If Blueprint.id is referenced by any Project (regardless of status) → soft-delete only: isDeleted=true. Hard delete is not permitted.
- A soft-deleted blueprint does not appear in quotation filter results. Existing quotations referencing it remain valid until project Start.

7.6.3 Phase Reordering

- Phase.order values must remain unique within a blueprint after reorder. The reorder operation MUST be submitted as the full ordered list, not individual swap operations, to avoid transient uniqueness violations.
- Reordering does NOT affect dependency declarations (dependsOnPhaseIds). Dependencies are by ID, not order.

7.6.4 Minimum Phase Requirement

- A blueprint with zero phases is not valid. Attempting to save a blueprint without at least one phase → 422 { code: 'BLUEPRINT_REQUIRES_PHASES' }.
- Attempting to delete the last phase of a blueprint → 422 { code: 'CANNOT_DELETE_LAST_PHASE' }.

7.6.5 Task Board Deletion

- A TaskBoard MUST NOT be deleted if it is assigned to any Phase (blueprint or ProjectPhase). Attempt → 422 { code: 'BOARD_IN_USE', phaseCount }.

8. Project Execution Module

8.1 Project Lifecycle FSM

Status	Description	Entry Condition	Exit Condition
PENDING_START	Proposal accepted; awaiting Start click	Quotation status = ACCEPTED	projects.start user clicks Start
ACTIVE	Project running	Start clicked	All phases COMPLETED
COMPLETED	All phases done, all milestones paid	Last phase approved by client	Automatic
ARCHIVED	Manually archived	COMPLETED or ACTIVE	Manual by projects.archive
CANCELLED	Cancelled before completion	Any active state	Manual by projects.archive

A project in PENDING_START does NOT count against the tenant's max_projects entitlement limit. It counts once status moves to ACTIVE.

8.2 Starting a Project [PK: projects.start]

76. POST /api/v1/projects/:id/start
77. Validate: project.status = PENDING_START, quotation.status = ACCEPTED.
78. Check tenant entitlement: active project count < max_projects.
79. Execute blueprint snapshot (Section 7.4).
80. Set Project.status = ACTIVE, Project.startedAt = now.
81. If paymentModel = ONE_TIME_BEFORE_START: immediately generate invoice for the single all-phase milestone and send to client.
82. Unlock initial phases: phases with no dependencies → set ProjectPhase.status = ACTIVE, ProjectPhase.unlockedAt = now.
83. Emit WebSocket event: project:started { projectId, startedAt } to tenant room.

8.3 Phase Lifecycle FSM

Status	Description	Transition Rule
PENDING	Waiting for dependencies	Auto-set at snapshot. Unlocks when all prerequisite phases reach COMPLETED.
ACTIVE	Work in progress	Set when unlocked. Tasks become accessible.
IN_REVIEW	All tasks done; pending approval	Auto-set when last task moves to Done column.
CLIENT_PENDING	Admin approved; awaiting client	Set by approvals.review user clicking Approve.
COMPLETED	Client approved	Set when client clicks Approve in portal.
ON_HOLD	Paused	Manual toggle by projects.* user. Resumes to ACTIVE.

8.3.1 Phase Unlock Logic

On every phase status change to COMPLETED, run the unlock check:

84. For each other phase P in the same project where P.status = PENDING:
85. Resolve P's prerequisites: all phaseIds in P.dependsOnPhaseIds (mapped to ProjectPhase rows).
86. If ALL prerequisites have status = COMPLETED → set P.status = ACTIVE, P.unlockedAt = now.
87. Emit WebSocket event: phase:unlocked { projectPhaseId, projectId }.

8.4 Phase Progress Computation

Phase progress is always computed on read — never stored:

```
progress(phase) = (tasks WHERE status='DONE' AND projectPhaseId=phase.id).count /
(tasks WHERE projectPhaseId=phase.id).count * 100
```

If a phase has 0 tasks: progress = 0.

Project-level progress = AVG(phase.progress) across all ProjectPhases for the project.

8.5 Task Board Behaviour

8.5.1 Task Movement

Tasks move through board columns via PATCH /api/v1/tasks/:id { columnId } [PK: tasks.manage].

- Task always belongs to exactly one ProjectPhase (projectPhaseId). This MUST NOT change after creation.
- Moving a task to the final 'Done' column sets Task.status = DONE. Moving it back sets Task.status = IN_PROGRESS.
- When last task in a phase moves to Done: auto-set ProjectPhase.status = IN_REVIEW (if no tasks remain non-Done).

8.5.2 Shared Board Behaviour

- A shared board renders tasks from all its assigned phases.
- Each task card MUST display a phase badge showing its ProjectPhase.name.
- Board provides a phase filter: 'All Phases' or a specific phase. Filtered view shows only tasks for the selected phase.
- Phase progress is computed per-phase regardless of board sharing.

8.6 Admin Approval Flow [PK: approvals.review]

88. Admin sees IN_REVIEW phases in the Approval Queue (GET /api/v1/approvals/queue).
89. POST /api/v1/approvals { projectPhaseId, decision: 'APPROVE' | 'REQUEST_CHANGES', comment? }
90. Write Approval row { projectPhaseId, reviewerId, decision, comment, timestamp: now, ipAddress }. Append-only — no update or delete endpoint.
91. If APPROVE: set ProjectPhase.status = CLIENT_PENDING. Push deliverables to client portal. Emit approval:submitted event. Notify client.
92. If REQUEST_CHANGES: set ProjectPhase.status = ACTIVE. Emit approval:submitted event. Task assignees notified with comment.

8.7 Client Feedback & Approval [Client actor]

8.7.1 Submit Feedback

POST /api/v1/feedback { projectPhaseId, content, attachments? } [Client actor]

93. Create ClientFeedback row.
94. Set ProjectPhase.status = ACTIVE (client rejected current deliverable).
95. Notify users with approvals.review permission.

8.7.2 Approve Phase

POST /portal/:token/approve { projectPhaseId } [Client actor]

96. Validate phase is in CLIENT_PENDING status.
97. Set ProjectPhase.status = COMPLETED, ProjectPhase.completedAt = now.
98. Run phase unlock logic (Section 8.3.1).
99. Run milestone completion check (Section 11.2).
100. Emit phase:completed { projectPhaseId, projectId }.

8.8 Project Execution Edge Cases

(Added for Engineering Completeness)

8.8.1 Concurrent Phase Approval

- If two users with approvals.review submit POST /approvals for the same projectPhaseId simultaneously: the first write completes. The second finds phase.status already transitioned and MUST return 422 { code: 'PHASE_ALREADY_REVIEWED', currentStatus }.
- Engineer note: use a database-level transaction with SELECT FOR UPDATE on the ProjectPhase row before writing the Approval record.

8.8.2 Task Assigned to Inactive User

- PATCH /tasks/:id { assignedTo: userId } where User.isActive=false → 422 { code: 'USER_INACTIVE' }. Existing tasks remain assigned to the user; the system does not auto-reassign on deactivation.

8.8.3 Moving Task from Done Back to In-Progress

- If the phase was auto-advanced to IN_REVIEW because all tasks were Done, and a task is subsequently moved back out of Done: phase.status MUST revert to ACTIVE. This check runs synchronously on task column update.

8.8.4 Approving a Phase with No Tasks

- A phase with zero tasks cannot auto-advance to IN_REVIEW (division-by-zero guard). A user with approvals.review may manually submit POST /approvals { projectPhaseId, decision: 'APPROVE' } for a zero-task phase. This is valid (e.g. a milestone-only phase with no task work).

8.8.5 Parallel Phase Completion Order

- Two parallel phases may complete in any order. The milestone completion check (Section 11.2) is triggered after every phase approval and checks all phases in the milestone — order is irrelevant.

8.8.6 Phase Deadline on Blueprint Without Duration

- If blueprint Phase.defaultDurationDays is null, ProjectPhase.deadline is null. Overtime alerts MUST NOT fire for phases without a deadline.

8.8.7 Archiving a Project

- PATCH /api/v1/projects/:id/archive [PK: projects.archive] is permitted when project.status is ACTIVE or COMPLETED. Transitions to ARCHIVED. Active BullMQ jobs for the project (pending invoice triggers) MUST be cancelled.
- An ARCHIVED project does NOT count against max_projects entitlement.

9. RBAC & Team Management Module

9.1 Permission Group Management [PK: permission-groups.manage]

9.1.1 Create Permission Group

POST /api/v1/permission-groups { name, description?, permissionKeys[] }

- name: 2–100 chars, unique per tenant.
- permissionKeys: each must exist in the global Permission catalog.
- isSystem=false, isEditable=true for user-created groups.

9.1.2 Update Permission Group

PATCH /api/v1/permission-groups/:id { name?, permissionKeys? }

- If group is Administrator (isSystem=true): MUST NOT remove users.manage, permission-groups.manage, or settings.manage from permissionKeys (Administrator protection rule).
- Re-compute effective permissions cache for all members immediately after save.

9.1.3 Delete Permission Group

DELETE /api/v1/permission-groups/:id

- If isSystem=true → 422 { code: 'CANNOT_DELETE_SYSTEM_GROUP' }.
- Cascade: remove all UserPermissionGroup rows for this group.

9.2 Default Permission Groups Bootstrap

These groups are created automatically at tenant bootstrap. All are isEditable=true except Administrator (isSystem=true).

Group	Key Permissions
Administrator	All keys (wildcard). isSystem=true. Cannot be deleted.
Sales	crm.view, crm.create, crm.edit, crm.delete, contacts.view, contacts.manage, quotations.view, quotations.create, quotations.send, projects.view
Operations	services.view, services.manage, packages.manage, blueprints.view, blueprints.manage, projects.view, tasks.view, tasks.manage, tasks.assign
Project Manager	projects.view, projects.start, projects.archive, tasks.view, tasks.manage, tasks.assign, approvals.review, contacts.view, analytics.view
Finance	invoices.view, invoices.create, invoices.record_payment, analytics.view, projects.view, change_requests.approve
Designer	tasks.view, tasks.manage, projects.view, notifications.manage
Developer	tasks.view, tasks.manage, projects.view, notifications.manage
Viewer	crm.view, projects.view, invoices.view, analytics.view, blueprints.view, services.view

9.3 User Invitation & Management

9.3.1 Invite User [PK: users.invite]

POST /api/v1/users/invite { email, name, permissionGroupIds[] }

101. Check email not already a user in this tenant.
102. Check adding user would not exceed max_users entitlement.
103. Create User { tenantId, email, name, isActive: true, passwordHash: null }.
104. Create UserPermissionGroup rows for each permissionGroupId.
105. Generate invite token (24h TTL). Email link to set password and activate account.

9.3.2 Assign Permission Groups [PK: users.manage]

POST /api/v1/users/:id/groups { permissionGroupIds[] } — replaces current group assignments.

- Run Administrator protection checks before saving.
- Invalidate perm:{tenantId}:{userId} cache.

9.3.3 Deactivate User [PK: users.manage]

PATCH /api/v1/users/:id { isActive: false }

- Run Administrator protection check: if user is last Administrator member → 422 { code: 'CANNOT_DEACTIVATE_LAST_ADMIN' }.
- Deactivated user's JWT tokens become invalid on next refresh (cannot get new access token).

9.4 Per-User Permission Overrides [PK: users.manage]

POST /api/v1/users/:id/overrides { permissionKey, effect: 'GRANT' | 'REVOKE' }

- Each permissionKey+userId pair is unique (upsert).
- Applied after group union in effective permissions resolution.
- Invalidate cache on save.

9.5 Workload View [PK: users.view]

GET /api/v1/team/workload — returns per-user active task summary. Computed on read:

```
workload[userId] = { count: tasks.count(WHERE assignedTo=userId AND status != 'DONE' AND project.status='ACTIVE'), tasks: [...task summaries] }
```

9.6 RBAC Edge Cases

(Added for Engineering Completeness)

9.6.1 User With No Permission Groups

- A user with no group memberships has an empty effective permission set. Every protected endpoint returns 403. This is a valid state (e.g. newly invited user before groups assigned).

9.6.2 Conflicting Overrides

- If a UserPermissionOverride { effect: GRANT } and a group-level revoke exist for the same key, the override takes precedence: GRANT override adds the key even if no group grants it; REVOKE override removes the key even if a group grants it.
- Two overrides for the same user + key MUST NOT be created. The endpoint is an upsert by (userId, permissionKey).

9.6.3 Permission Group With No Permissions

- A PermissionGroup with an empty permission set is valid. Members of such a group gain no permissions from it (but may gain permissions from other groups).

9.6.4 User Assigned to Same Group Twice

- POST /users/:id/groups is a replace operation (full list). The system MUST deduplicate permissionGroupIds before writing. Duplicate groupIds in the input array → silently deduplicated, not an error.

9.6.5 Inviting an Existing User

- If the invited email already belongs to an active User in this tenant → 422 { code: 'USER_ALREADY_EXISTS' }. If the email belongs to a deactivated user → 422 { code: 'USER_DEACTIVATED_EXISTS' } with hint to reactivate.

10. Client Portal Module

10.1 Portal Access Model

The client portal is a completely separate API surface, prefixed `/portal/:token/...` or authenticated via a Client JWT. Client actions do NOT use tenant RBAC. The portal is scoped to a specific `contactId` + `tenantId` pair.

10.2 Portal Project List [Client actor]

GET `/portal/projects` — returns all projects where `project.contactId = authenticatedContactId` AND `project.tenantId = tenantId`.

- Returns: `projectId`, `name`, `status`, `overall progress (computed)`, `phase summary`.
- Projects in `PENDING_START` status: client sees an 'Awaiting Start' holding state — no task detail.

10.3 Proposal Review & Amendment [Client actor]

Described in Section 6.6.3 (amendment), 6.6.4 (accept), 6.6.5 (reject).

Client can view proposal PDF inline in the portal. All versions are accessible for reference; only the latest SENT version is actionable.

10.4 Project Tracking [Client actor]

Only accessible after `Project.status = ACTIVE` (i.e., after `projects.start` user has clicked Start).

- GET `/portal/projects/:id` — returns: `overall progress`, `phase list with statuses` and `per-phase progress`, `milestone summary with completion` and `invoice status`.
- Phase status visible to client: `Completed` | `Active` | `Pending` (not `IN_REVIEW` or `CLIENT_PENDING` — those appear as 'In Review').

10.5 Deliverable Viewing & Feedback [Client actor]

GET `/portal/projects/:id/phases/:phaseId/deliverables` — returns list of assets in `CLIENT_PENDING` or `COMPLETED` phases.

- Assets are served via presigned S3 URLs, TTL: 1 hour.
- Supported inline preview: PDF, JPEG, PNG, WebP, MP4.
- Other file types: download-only (no inline preview).

10.6 New Project Request [Client actor]

POST `/portal/project-requests` { `description?`, `serviceSuggestions?` }

Creates a lightweight record visible to users with `projects.view`. It is NOT a project. The agency must respond with a quotation. The request cannot self-activate.

10.7 Invoice Access [Client actor]

GET /portal/invoices — returns all invoices for client's projects on this tenant.

- Invoice PDF: presigned S3 URL, TTL 1 hour.
- Client can view invoice status (UNPAID | PARTIALLY_PAID | PAID | OVERDUE) but cannot record payments.

10.8 Client Portal Edge Cases

(Added for Engineering Completeness)

10.8.1 Client Accessing Deleted Project

- If a project is ARCHIVED or CANCELLED, the client portal still shows it in the project list (read-only). No actions are available. Status label shows 'Completed' or 'Closed' as appropriate.

10.8.2 Proposal Amendment After Prior Rejection

- A client MUST NOT be able to amend a proposal that is in REJECTED status. The POST /proposals/:id/amend endpoint MUST check status ∈ {SENT, AMENDMENT_REQUESTED} before accepting. Otherwise → 422 { code: 'PROPOSAL_NOT_ACTIONABLE', status }.

10.8.3 Client Viewing Proposal for Non-Started Project

- A client can view their accepted proposal at any time, even after project start. The proposal is read-only post-acceptance; amendment and accept/reject buttons are not shown.

10.8.4 Presigned URL Expiry

- If a client opens a deliverable URL that has expired (1-hour TTL), the system MUST regenerate a fresh presigned URL on the next GET /portal/phases/:id/deliverables/:assetId/url call. The client-facing experience should be seamless.

11. Invoicing & Payment Automation Module

11.1 Invoice Source Types

An Invoice has exactly one source — never both, never neither:

Source	milestoneId	changeRequestId	Trigger
Milestone invoice	SET	NULL	BullMQ job on milestone completion
Change request invoice	NULL	SET	On ChangeRequest status → APPROVED
One-time before start	SET (single milestone)	NULL	On project Start click

11.2 Milestone Completion Trigger [SYS]

This logic runs after every client phase approval (Section 8.7.2):

106. Find the milestone M that contains the just-approved ProjectPhase (via MilestonePhase).
107. Check if ALL other ProjectPhases in M have status = COMPLETED.
108. If NO → no action. Milestone waits.
109. If YES → check milestone firing order: verify all milestones with a lower order that M depends on (derived from phase-level DAG) have already fired (Invoice generated for them).
110. If firing order not satisfied → wait (should not normally occur; log warning).
111. If satisfied → enqueue BullMQ job: generate-invoice { milestoneId }.

11.2.1 BullMQ Job: generate-invoice

112. Create Invoice { projectId, milestoneId, amount: milestone.amount, amountPaid: 0, status: UNPAID, dueDate: now + 30d (configurable), sentAt: now }.
113. Generate PDF (branded template). Store in S3: tenant/{tenantId}/invoices/{invoiceId}/invoice.pdf.
114. Email invoice PDF to Contact.email.
115. Create portal Notification for client.
116. Emit WebSocket event: invoice:generated { invoiceId, milestoneId, amount, dueDate }.

11.3 One-Time Payment Models [SYS]

- ONE_TIME_BEFORE_START: invoice generated immediately when projects.start user clicks Start. Milestone triggerType=ON_PROJECT_START.
- ONE_TIME_AFTER_FINISH: invoice generated when last phase of project reaches COMPLETED. Milestone triggerType=ON_COMPLETION. Standard milestone completion trigger applies.

11.4 Partial Payments [PK: invoices.record_payment]

POST /api/v1/invoices/:id/payments { amount, paidAt, method }

- Validate: amount > 0, amount ≤ (invoice.amount - invoice.amountPaid).
- Create Payment { invoiceId, amount, paidAt, method }.
- invoice.amountPaid = SUM(payments.amount) — recomputed, never stored directly.
- invoice.status (derived, not stored): UNPAID (amountPaid=0, not overdue), PARTIALLY_PAID (0 < amountPaid < amount), PAID (amountPaid ≥ amount), OVERDUE (any of the above + dueDate < now AND not PAID).

11.5 Invoice PDF Export [PK: invoices.view]

GET /api/v1/invoices/:id/pdf → presigned S3 URL (TTL 1h). PDF is pre-generated at invoice creation; regenerated on demand if S3 object missing.

11.6 Overtime Alert [SYS]

Scheduled BullMQ job (runs hourly): for each ProjectPhase WHERE deadline IS NOT NULL AND deadline < now AND status NOT IN [COMPLETED] → create/update Notification for users with projects.view, fire WebSocket notification:new event.

Overtime is not automatically converted to a change request — it requires manual entry (Section 12).

11.7 Invoice & Payment Edge Cases

(Added for Engineering Completeness)

11.7.1 Invoice Generation Failure

- If the generate-invoice BullMQ job fails (PDF generation error, email error): the job retries per queue policy (3 attempts, exponential backoff). If all retries fail: job moves to dead-letter queue. An alert fires to the monitoring stack. The milestone is NOT re-triggered automatically — manual intervention is required.
- Engineer note: invoice row SHOULD be created in the first step of the job (before PDF generation) so the billing record exists even if email fails. PDF generation and email are subsequent steps.

11.7.2 Overpayment

- If a Payment is recorded that would cause amountPaid > invoice.amount → 422 { code: 'OVERPAYMENT', maximum: invoice.amount - invoice.amountPaid }. Overpayments are not accepted in v1.

11.7.3 Invoice for Cancelled/Archived Project

- Invoices for ARCHIVED or CANCELLED projects remain valid and payable. Archiving a project does NOT void its invoices.

11.7.4 Duplicate Milestone Invoice

- BullMQ job deduplication (job ID = 'invoice:{milestoneId}') prevents duplicate invoice creation. If a job with the same ID is already queued or completed → skip enqueue. Additionally, before creating Invoice, check: if Invoice WHERE milestoneId = :id already exists → skip creation, log warning.

11.7.5 OVERDUE Status Derivation

- OVERDUE applies when: `invoice.dueDate < now` AND `invoice.status ∈ {UNPAID, PARTIALLY_PAID}`. PAID invoices are never OVERDUE even if `dueDate` is past. Status is computed on every read — it is not stored in the database.

12. Change Requests Module

12.1 Change Request Lifecycle FSM

Status	Description	Transition
REQUESTED	Submitted, awaiting review	→ APPROVED (change_requests.approve), → REJECTED (change_requests.approve)
APPROVED	Approved; invoice auto-generated	→ INVOICED (system, on invoice create)
REJECTED	Declined by admin	Terminal state
INVOICED	Invoice created and sent	Terminal state

12.2 Create Change Request [PK: change_requests.manage]

POST /api/v1/projects/:id/change-requests { type: 'SCOPE' | 'OVERTIME', description, amount }

- amount: ≥ 0 , max 2 decimal places.
- Client may also submit via portal (no permission key needed for portal client).
- Creates ChangeRequest { projectId, type, description, amount, status: REQUESTED, createdBy: userId (or contactId for client) }.
- Notify users with change_requests.approve permission.

12.3 Approve / Reject [PK: change_requests.approve]

POST /api/v1/change-requests/:id/approve | /reject { comment? }

On Approve:

117. Set ChangeRequest.status = APPROVED, approvedBy = userId.
118. Create Invoice { projectId, changeRequestId, milestoneId: null, amount: changeRequest.amount, status: UNPAID, dueDate: now+30d }.
119. Set ChangeRequest.status = INVOICED.
120. Generate PDF, email client, portal notification.

On Reject:

121. Set ChangeRequest.status = REJECTED, rejectedBy = userId.
122. Notify change request creator.

Rejection is terminal — a rejected CR cannot be reopened. A new CR must be submitted.

12.4 Change Request Edge Cases

(Added for Engineering Completeness)

12.4.1 Change Request on Completed Project

- A ChangeRequest MAY be submitted on a project with status=COMPLETED (e.g. late-billed overtime). The endpoint does not restrict by project status.
- A ChangeRequest MUST NOT be submitted on a project with status=ARCHIVED or CANCELLED → 422 { code: 'PROJECT_NOT_ACTIVE' }.

12.4.2 Multiple Pending Change Requests

- Multiple ChangeRequests in REQUESTED status can exist simultaneously for the same project. Each follows its own approval lifecycle independently.

12.4.3 Zero-Amount Change Request

- A ChangeRequest with amount=0 is not meaningful and MUST be rejected → 422 { code: 'CHANGE_REQUEST_AMOUNT_REQUIRED', min: 0.01 }.

13. Notification & Automation Module

13.1 Notification Triggers

Event	Recipients	Channel
Lead assigned	Assigned user	In-app
Proposal sent to client	Client	Email + portal
Client amendment request	Users with quotations.create	In-app + email
Proposal accepted	Users with projects.start	In-app + email
Project started	All project team	In-app
Phase unlocked	Assigned task owners of that phase	In-app
Task assigned	Assigned user	In-app
Deliverable in review (IN_REVIEW)	Users with approvals.review	In-app + email
Admin approved, awaiting client	Client	Portal + email
Client feedback submitted	Users with approvals.review	In-app + email
Client phase approved	Project team + users with projects.view	In-app
Invoice generated	Client (portal + email), users with invoices.view	Email + portal + in-app
Payment recorded	Users with invoices.view	In-app
Phase overdue	Users with projects.view	In-app + email
Change request submitted	Users with change_requests.approve	In-app + email
Change request approved/rejected	Creator of change request	In-app + email

13.2 WebSocket Events

All events are emitted to the tenant's Socket.io room: tenant:{tenantId}. Client portal events additionally emitted to client:{contactId}:{tenantId}.

13.3 Email Queue (BullMQ)

- All emails are dispatched via BullMQ queue: email-dispatch.
- Retry policy: 3 attempts, exponential backoff (1m, 5m, 15m).
- Failed after all retries: log to dead-letter queue, alert monitoring.
- Email templates are per event type; support Arabic and English based on recipient preference.

13.4 Notification Preferences [Tenant User]

PATCH /api/v1/notifications/preferences — user toggles email and in-app notification on/off per event type. Stored per user. System respects preferences when dispatching (except critical security emails which are always sent).

13.5 Notification Edge Cases

(Added for Engineering Completeness)

13.5.1 Notification Fan-Out Limits

- For events that target 'users with permission X', the fan-out query is: `SELECT userId FROM effectivePermissions WHERE permissionKey = X AND tenantId = :id`. On large tenants with many users, this could be a high fan-out. Engineer note: consider batching in-app notification inserts above 50 recipients.

13.5.2 Notification for Deactivated Users

- In-app notifications **MUST NOT** be created for users with `isActive=false`. Email notifications **MUST NOT** be sent to deactivated users. The fan-out query **MUST** include `AND isActive=true`.

13.5.3 Stale Notification Links

- Notifications contain a link to the relevant entity (e.g. `/projects/:id`). If the entity is subsequently archived or deleted, the link may return 404. The client should handle 404 gracefully from notification deep-links — this is expected behaviour in v1.

14. Analytics Module

14.1 Tenant Analytics [PK: analytics.view]

GET /api/v1/analytics/sales | /delivery | /finance — all computed on read from live data. No pre-aggregated tables in v1.

Metric	Computation	Endpoint
Lead conversion rate	COUNT(leads WHERE conversionStatus=CONVERTED) / COUNT(leads WHERE tenantId) * 100	/analytics/sales
Proposal win rate	COUNT(proposals WHERE status=ACCEPTED) / COUNT(proposals WHERE status IN [ACCEPTED,REJECTED,SENT]) * 100	/analytics/sales
Overdue projects	COUNT(projectPhases WHERE deadline < now AND status NOT IN [COMPLETED])	/analytics/delivery
Active projects by status	COUNT(projects) GROUP BY status WHERE tenantId	/analytics/delivery
Revenue	SUM(payments.amount WHERE project.tenantId=tenantId)	/analytics/finance
Outstanding invoices	SUM(invoice.amount - invoice.amountPaid WHERE status IN [UNPAID, PARTIALLY_PAID, OVERDUE])	/analytics/finance

14.2 Platform Analytics [Platform Owner]

GET /api/v1/platform/analytics

- MRR = SUM(plan.price WHERE tenant.subscriptionStatus=ACTIVE AND plan.billingPeriod=MONTHLY) + SUM(plan.price/12 WHERE billingPeriod=ANNUAL).
- Plan distribution: COUNT(tenants) GROUP BY planId.
- Active tenants last 30 days: tenants with at least one active project in last 30 days.

14.3 TransitionLog (Future Analytics Foundation)

Written on every lead stage change and every phase status change:

```
TransitionLog { id, tenantId, entityType: LEAD|PHASE, entityId, fromState, toState,
  at: timestamp, byUserId }
```

In v1: no analytics UI reads this. In v2+: pipeline velocity (avg days per sales stage), project completion time, phase bottleneck (avg time-in-phase per service type).

Why capture now?

Historical transition data cannot be backfilled. If logging starts in v2, all v1 projects will have no history. The log is append-only. There is no update or delete endpoint for TransitionLog rows. Storage cost is negligible relative to the irreplaceable analytical value.

14.4 Analytics Edge Cases

(Added for Engineering Completeness)

14.4.1 Division-by-Zero Guards

- Lead conversion rate: if total leads = 0 → return 0, not NaN or error.
- Proposal win rate: if sent proposals = 0 → return 0.
- Phase progress: if phase task count = 0 → return 0 (see Section 8.4).

14.4.2 Analytics Performance

- v1 analytics are computed on read. For tenants with large datasets, these queries may be slow. Engineer note: add database indexes on: Lead.conversionStatus, Proposal.status, ProjectPhase.deadline, Invoice.status, Payment.invoiceId.
- If any analytics endpoint exceeds 2s response time in staging, move to materialized view or background-computed cache before launch.

15. Platform Administration Module

15.1 Platform Owner Capabilities

The Platform Owner sits above all tenants and has no access to any tenant's operational data. Separation is enforced at the query level — no Platform Owner API endpoint can query tenant-scoped operational tables (Project, Task, Client, etc.).

15.2 Plan Management [Platform Owner]

POST/PATCH /api/v1/platform/plans { name, price, currency: 'SAR', billingPeriod, isActive }

- Retiring a plan: set isActive=false. Existing tenants on the plan keep it; no new signups.
- Price update: affects new subscriptions immediately. Existing subscriptions: governed by billing provider (Stripe).

15.3 Entitlement Management [Platform Owner]

POST/PATCH /api/v1/platform/plans/:id/entitlements { key, value }

- Entitlement keys: max_projects, max_users, storage_gb, white_label (bool), custom_domain (bool), api_access (bool), advanced_reporting (bool).
- Changing an entitlement takes effect immediately for all tenants on that plan (Redis cache invalidated).

15.4 Tenant Lifecycle [Platform Owner]

Action	Endpoint	Effect
Provision	POST /api/v1/tenants	Runs bootstrap sequence (Section 3.2).
Suspend	PATCH /api/v1/tenants/:id/suspend	Sets subscriptionStatus=SUSPENDED. All tenant API requests → 403.
Reactivate	PATCH /api/v1/tenants/:id/reactivate	Sets subscriptionStatus=ACTIVE. API access restored.
View status	GET /api/v1/tenants/:id	Returns plan, status, user count, project count. No operational data.

15.5 Platform Owner Edge Cases

(Added for Engineering Completeness)

15.5.1 Downgrading a Tenant Plan

- If a tenant is downgraded to a plan with lower max_users or max_projects: existing users and projects are NOT auto-deleted. The tenant is simply over-limit.
- Over-limit tenants: new user invitations and project creations are blocked. Existing data is preserved. The tenant must self-resolve by deactivating users or completing/archiving projects.

15.5.2 Suspending a Tenant with Active BullMQ Jobs

- When a tenant is suspended: all pending BullMQ jobs for that tenantId MUST be paused (not deleted). Jobs resume when tenant is reactivated. Engineer note: use BullMQ's queue-pause-by-pattern if available; otherwise, check tenant suspension status at job start and re-queue with delay.

15.5.3 Plan with No Entitlements

- A plan with no Entitlement rows is treated as having all limits at zero (no projects, no users, zero storage). This is intentional for the Platform Owner to define a 'suspended capability' plan. Engineers MUST handle null/missing entitlement values as zero, not as unlimited.

16. Non-Functional Specifications

16.1 API Response Standards

Code	Meaning	Payload Shape
200	Success (GET/PATCH)	{ data: {...} }
201	Created (POST)	{ data: {...} }
400	Bad request (missing field)	{ code, message, field? }
401	Unauthenticated	{ code: 'UNAUTHENTICATED' }
403	Forbidden	{ code: 'FORBIDDEN', requiredPermission? }
404	Not found	{ code: 'NOT_FOUND', resource }
422	Business rule violation	{ code, message, details? }
429	Rate limited	{ code: 'RATE_LIMITED', retryAfter }
500	Internal error	{ code: 'INTERNAL_ERROR', requestId }

16.2 Validation Standards

- All inputs validated with Zod schemas before reaching service layer.
- Validation errors return 422 with field-level detail: { code: 'VALIDATION_ERROR', errors: [{ field, message }] }.
- Monetary amounts: stored as DECIMAL(12,2) in database. Never use floating point.
- Dates/times: stored as UTC timestamps. Returned as ISO 8601.

16.3 Performance Targets

Metric	Target
GET endpoint p95 response	< 200ms
POST/PATCH p95 response	< 500ms
WebSocket event latency	< 500ms end-to-end
Invoice PDF generation	< 5 seconds (BullMQ async)
Blueprint snapshot-on-Start	< 2 seconds (in transaction)
Effective permissions cache hit	> 95% of requests

16.4 Security Requirements

- JWT: RS256 algorithm. Public key distributed to services. Private key in secrets manager.
- Passwords: bcrypt with cost factor 12, or argon2id.
- RBAC guards: applied at NestJS controller level via @UseGuards(PermissionGuard('key')). Also enforced at service layer for internal calls.

- S3 presigned URLs: TTL 1 hour for assets, 1 hour for invoice PDFs. No direct bucket access.
- All API inputs sanitized to prevent XSS/injection before any processing.
- Rate limiting: POST /auth/* endpoints: 10 requests/minute per IP.

16.5 i18n

- All UI strings externalized via i18next.
- Arabic: IBM Plex Sans Arabic (open source). RTL layout enforced.
- English: Saudi Air (pending licence confirmation — see PRD open item F). Fallback: Inter.
- Language preference: stored per user. Default: Arabic.
- Email templates: bilingual versions per event type.

16.6 Audit & Immutability

- Approval records (Approval entity): no UPDATE or DELETE endpoint exists. Append-only by design.
- TransitionLog: no UPDATE or DELETE endpoint exists. Append-only by design.
- Payment records (Payment entity): no UPDATE or DELETE endpoint. Correction via new offsetting payment.
- AuthToken: usedAt is set once and never cleared.

16.7 Concurrency & Locking Strategy

(Added for Engineering Completeness)

The following table defines locking requirements per critical operation.

Operation	Locking Strategy	Rationale
Phase status transition	SELECT FOR UPDATE on ProjectPhase row	Prevents concurrent review decisions on same phase.
Invoice generation trigger	BullMQ job deduplication by milestoneId	Prevents duplicate invoice for same milestone.
AuthToken redemption	UPDATE WHERE usedAt IS NULL (atomic CAS)	Prevents replay — only one redemption can set usedAt.
Admin protection check	SELECT FOR UPDATE on PermissionGroup/UserPermissionGroup	Prevents race that would remove last admin.
Project start	Transaction with SELECT FOR UPDATE on Project	Prevents double-start from two concurrent requests.
Milestone amount validation	Validated at save time within transaction	Snapshot prevents post-save drift.
Plan entitlement check	Cache read + DB fallback; no hard lock	Occasional over-limit race acceptable in v1; log and alert.

16.8 Audit Log Coverage

(Added for Engineering Completeness)

The following events **MUST** be written to an audit trail (immutable, append-only). The audit log is separate from TransitionLog (which tracks state changes only).

Event	Audit Entity	Fields Captured
Phase approval / rejection	Approval	reviewerId, decision, comment, timestamp, ipAddress
Project Start	AuditLog entry (or system note on Project)	startedBy, startedAt, blueprintId snapshot reference
Invoice generation	Invoice.sentAt + BullMQ job log	milestoneId, amount, triggeredAt
Change request approval/rejection	ChangeRequest.approvedBy / rejectedBy + timestamp	actorId, decision, timestamp
Permission Group changes	AuditLog (if implemented)	changedBy, groupId, before/after permissionKeys
Admin protection violation attempt	System log (at minimum)	actorId, attempted action, timestamp
Tenant suspension / reactivation	Platform-level audit log	operatorId, tenantId, action, timestamp

16.9 Resilience & Graceful Degradation

(Added for Engineering Completeness)

- Redis unavailable: RBAC permission checks **MUST** fall back to a direct database query. Do not deny all requests when Redis is down. Log cache miss.
- BullMQ broker unavailable: API endpoints that enqueue jobs (invoice triggers, email dispatch) **MUST** succeed for the primary write (Invoice row creation) and enqueue asynchronously with retry. If the broker is down, the job is persisted to a fallback table and retried on reconnect.
- S3 unavailable: file upload endpoints return 503 { code: 'STORAGE_UNAVAILABLE' }. Do not partially save asset metadata without confirming S3 write success.
- Email provider unavailable: BullMQ retry policy (3 attempts, exponential backoff) covers transient failures. If all retries fail, invoice/notification data is already persisted — the email is the only missing step. Dead-letter queue alerts operators.

17. End-to-End Data Flow Summary

17.1 Lead-to-Project Flow

Step	Actor	Write	Read / Trigger
New lead	Tenant User [crm.create]	Lead, TransitionLog	—
Stage movement	Tenant User [crm.edit]	Lead.stageId, TransitionLog	WebSocket notification
Conversion YES	Tenant User [crm.edit]	AuthToken (ONBOARDING)	Email → client
Client onboarding	Client	Contact.accountCreatedAt, AuthToken.usedAt	Portal provisioned
Quotation created	Tenant User [quotations.create]	Quotation, QuotationMilestone, MilestonePhase	—
Proposal sent	Tenant User [quotations.send]	Proposal.status=SENT	Email → client
Client accepts	Client	Proposal.status=ACCEPTED, Quotation.status=ACCEPTED	Notify projects.start users
Project started	Tenant User [projects.start]	Project, ProjectPhase rows (snapshot), Task rows (templates)	Invoice if ONE_TIME_BEFORE_START

17.2 Phase Execution Flow

Step	Actor	Write	Trigger
Task work	Team [tasks.manage]	Task.status, Task.columnId	WebSocket: task:updated
All tasks done	[SYS]	ProjectPhase.status=IN_REVIEW	Notify approvals.review users
Admin approves	Tenant User [approvals.review]	Approval, ProjectPhase.status=CLIENT_PENDING	WebSocket + client notification
Client approves	Client	ProjectPhase.status=COMPLETED, ProjectPhase.completedAt	Milestone check, phase unlock
Milestone complete	[SYS - BullMQ]	Invoice, Payment stub	Email + WebSocket: invoice:generated
Next phase unlocked	[SYS]	ProjectPhase.status=ACTIVE, unlockedAt	WebSocket: phase:unlocked

17.3 Transaction Boundaries

(Added for Engineering Completeness)

The following operations **MUST** execute within a single database transaction. Partial completion is a defect.

Operation	Transaction Scope
Tenant bootstrap (Section 3.2)	Create Tenant + User + default PermissionGroups + UserPermissionGroup (5 writes)
Project Start (Section 8.2)	Snapshot ProjectPhase rows + create Task rows from templates + set Project.status=ACTIVE
Milestone completion + invoice enqueue	Confirm all phases COMPLETED + enqueue BullMQ job (job creation is transactional in BullMQ v3+)
Stage deletion with cascade (Section 5.4.5)	Delete SalesFlowStage + update Lead.stageId for all affected leads
Change request approval + invoice create (Section 12.3)	Set ChangeRequest.status=APPROVED + create Invoice + set ChangeRequest.status=INVOICED
AuthToken redemption (Section 2.3)	Set AuthToken.usedAt + create/update Contact + issue session
Permission group delete cascade (Section 9.1.3)	Delete PermissionGroup + delete all UserPermissionGroup rows + invalidate caches

18. Error Code Catalogue

All business-rule error responses return HTTP 422 with the following codes. Authentication errors return 401; authorization errors return 403.

Code	Trigger Condition
INVALID_CREDENTIALS	Login: user not found, inactive, or password mismatch.
INVALID_REFRESH_TOKEN	Refresh: token not found, expired, or revoked.
TENANT_NOT_FOUND	Subdomain/slug resolves to no tenant.
TENANT_SUSPENDED	Tenant subscriptionStatus = SUSPENDED.
FORBIDDEN	Caller lacks required permission key.
PLAN_LIMIT_EXCEEDED	Creating entity would exceed plan limit (max_projects, max_users, storage_gb).
DUPLICATE_SERVICE_NAME	Service name already exists in tenant.
SERVICE_IN_USE	Attempting to delete a service referenced by blueprints or packages.
BLUEPRINT_NOT_FOUND	No blueprint exactly matches selected service set.
CIRCULAR_PHASE_DEPENDENCY	Blueprint phase dependency graph contains a cycle.
MILESTONE_NOT_DEPENDENCY_CLOSED	Milestone grouping violates dependency-closed rule.
PHASE_NOT_IN_MILESTONE	A blueprint phase is not assigned to any milestone.
PHASE_IN_MULTIPLE_MILESTONES	A phase assigned to more than one milestone.
MILESTONE_AMOUNTS_DONT_SUM	Sum of milestone amounts $\neq$ quotation total.
ADMIN_PROTECTION_VIOLATION	Attempt to delete system group, remove last admin, deactivate last admin, or remove critical permissions.
CANNOT_DELETE_SYSTEM_GROUP	Attempt to delete isSystem=true PermissionGroup.
CANNOT_DEACTIVATE_LAST_ADMIN	Attempt to deactivate the sole Administrator group member.
AUTH_TOKEN_INVALID	AuthToken not found, already used, or expired.
VALIDATION_ERROR	One or more input fields fail Zod schema validation.
CHANGE_REQUEST_NOT_APPROVABLE	Attempt to approve/reject a CR not in REQUESTED status.
PROJECT_NOT_STARTABLE	Attempt to start a project not in PENDING_START status.

18.2 Error Response Schema

(Added for Engineering Completeness)

All error responses MUST conform to the following JSON schema. No endpoint may return a non-conforming error body.

Field	Type	Always Present	Description
code	string	Yes	Machine-readable error code from the catalogue (Section 18 and Section 18.3).
message	string	Yes	Human-readable description. May be shown in developer tooling, not end-user UI.
requestId	string (uuid)	Yes	Unique ID for this request. Used for log correlation. Generated per request by middleware.
field	string?	No	For VALIDATION_ERROR: the specific field that failed.
details	object?	No	Additional structured context (e.g. current, max for PLAN_LIMIT_EXCEEDED).
errors	array?	No	For VALIDATION_ERROR with multiple fields: array of { field, message } objects.

18.3 Additional Error Codes

(Added for Engineering Completeness)

Code	Trigger Condition
PACKAGE_CONTAINS_ARCHIVED_SERVICE	Package selection in quotation builder contains an archived service.
PACKAGE_TOO_FEW_SERVICES	Attempt to create package with fewer than 2 services.
LEAD_ALREADY_CONVERTED	Attempt to convert a lead that is already CONVERTED.
STAGE_HAS_ACTIVE_LEADS	Attempt to delete a stage containing leads without force:true.
QUOTATION_NOT_EDITABLE	Attempt to PATCH a quotation in SENT or ACCEPTED status.
BLUEPRINT_DELETED	Attempt to start a project whose blueprint has been soft-deleted.
PROPOSAL_NOT_ACTIONABLE	Client attempts to amend or accept/reject a proposal not in a valid state for that action.
PHASE_ALREADY_REVIEWED	Concurrent approval attempt on a phase that has already been reviewed.
USER_INACTIVE	Action targeting a deactivated user (assignment, approval, invitation).
USER_ALREADY_EXISTS	Invite with email already belonging to an active user in this tenant.
USER_DEACTIVATED_EXISTS	Invite with email of a deactivated user in this tenant.
CANNOT_DELETE_LAST_PHASE	Attempt to delete the only phase in a blueprint.
BLUEPRINT_REQUIRES_PHASES	Attempt to save a blueprint with zero phases.
BOARD_IN_USE	Attempt to delete a TaskBoard that is assigned to one or more phases.
OVERPAYMENT	Payment amount would cause amountPaid to exceed invoice.amount.
CHANGE_REQUEST_AMOUNT_REQUIRED	ChangeRequest submitted with amount ≤ 0 .
PROJECT_NOT_ACTIVE	ChangeRequest submitted on ARCHIVED or CANCELLED project.

Code	Trigger Condition
DUPLICATE_CONTACT_EMAIL	Attempt to create Contact with email already in use within same tenant.
STORAGE_UNAVAILABLE	S3 is unreachable during file upload (503).

19. Future Engineering Specifications (Out of Scope)

The documents listed below are intentionally maintained separately from this Functional Specification. Each concerns a distinct engineering concern that would increase the size and complexity of this document without improving its primary purpose as a behavioral contract.

These documents evolve independently as the build progresses. They reference this FS as their authoritative behavioral source but do not duplicate its content.

19.1 Entity Specification

Purpose: Defines every entity, field, lifecycle, constraints, relationships, indexes, defaults, nullable rules, and ownership. Derived from the data model in Section 9 of this FS but expanded to include all field-level constraints, index definitions, default value expressions, and entity lifecycle notes (created / updated / soft-deleted timestamps, UUID strategy).

19.2 Database Design Specification

Purpose: Defines the relational database architecture including the Entity-Relationship Diagram (ERD), index strategy, foreign key declarations, cascade rules, normalization decisions, soft-delete approach, UUID generation strategy (UUIDv7 recommended for time-ordering), and Prisma migration guidelines. Includes tenant-scoping enforcement patterns at the ORM layer.

19.3 API Specification

Purpose: Defines every endpoint with complete request/response contracts, full payload schemas (Zod-compatible), pagination strategy (cursor-based), filtering and sorting parameters, example request/response pairs, versioning policy (/api/v1 prefix), and deprecation notice protocol. Intended as the primary reference for frontend engineers and external API consumers.

19.4 Technical Architecture Specification

Purpose: Defines module boundaries (NestJS module map), inter-service interaction patterns, event architecture (WebSocket event catalog + BullMQ queue definitions), caching strategy (Redis key namespacing, TTL policy, invalidation triggers), file storage architecture (S3 bucket layout, lifecycle policies, presigned URL TTL governance), deployment topology (AWS ECS or equivalent, region selection, scaling policy), and infrastructure-as-code approach.

19.5 Design System Specification

Purpose: Defines reusable UI components, design tokens (color, spacing, typography, shadow, radius), iconography standard, accessibility requirements (WCAG 2.1 AA), RTL layout rules for Arabic, interaction states (hover, active, disabled, loading, error), and component library governance across all Buslahub products. Shared across Dalil and future products under the buslahub.com umbrella.

19.6 UX Flow Documentation

Purpose: Defines user journeys (happy path and alternate paths), navigation flows, screen-to-screen transitions, loading and error states, empty states, and interaction logic for every screen group defined in PRD Section 8. Complements the screen inventory (60 screens) with annotated wireframes and user-flow diagrams.

19.7 Sequence Diagram Collection

Purpose: Visualizes complex multi-actor workflows as sequence diagrams for engineering comprehension. Key sequences include: magic link onboarding, exact-match blueprint filtering, blueprint snapshot-on-Start, milestone invoice trigger chain, client amendment loop, RBAC permission resolution, and Platform Owner tenant lifecycle operations.

19.8 Testing Specification

Purpose: Defines acceptance criteria per module, unit testing strategy (Jest — target >80% service layer coverage), integration testing approach (module boundaries), E2E scenario catalog (Playwright — critical paths per PRD Section 3.3 success metrics), regression suite definition, and QA validation rules derived directly from the behavioral contracts in this FS. Test IDs cross-reference FS section numbers for traceability.

Companion Document Principle

This Functional Specification defines WHAT the system does and HOW each behavior is triggered, validated, and enforced.

Companion documents define HOW TO BUILD IT (architecture, schema, API contracts, UI) and HOW TO VERIFY IT (testing spec).

No companion document should replicate behavioral rules from this FS. They reference it by section number.

When a behavioral rule changes, it changes here first — companion documents are updated to reflect the change.